

Trilha de Docker

*Módulo 05: Criando e
Parametrizando Dockerfile.
docker-compose.yml e nginx.conf*

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Criando e Parametrizando Dockerfile, docker-compose.yml e nginx.conf

Conceito Geral

Em projetos Docker reais, normalmente utilizamos **três arquivos principais**:

- **Dockerfile** → define como a imagem da aplicação é construída
- **docker-compose.yml** → orquestra múltiplos contêineres
- **nginx.conf** → configura o servidor web e o proxy reverso

Esses arquivos trabalham juntos para criar um ambiente completo e padronizado.

1. Dockerfile

O **Dockerfile** define a imagem do ambiente onde a aplicação será executada.

Exemplo básico (Node.js)

```
FROM node:18

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

RUN npm run build
CMD ["npm", "start"]
```

Explicação das instruções

- **FROM** → define a imagem base
 - **WORKDIR** → define o diretório de trabalho no contêiner
 - **COPY** → copia arquivos do host para o contêiner
 - **RUN** → executa comandos durante a construção da imagem
 - **CMD** → comando executado quando o contêiner inicia
-

2. docker-compose.yml

O **docker-compose.yml** permite definir e executar **múltiplos contêineres** com um único comando.

Exemplo básico

```
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./app:/app
    depends_on:
      - db

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: app_db
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Principais conceitos

- **services** → define os contêineres da aplicação
- **ports** → mapeia portas do host para o contêiner
- **volumes** → garante persistência de dados
- **depends_on** → define a ordem de inicialização dos serviços

ANALOGIA: O docker-compose.yml é como um shopping: ele organiza várias lojas (serviços). Cada loja é aberta a partir de uma imagem, que por sua vez foi criada a partir de um Dockerfile.

Dockerfile → Imagem → Contêiner

3. nginx.conf

O **Nginx** atua como servidor web e **proxy reverso**, ficando na frente da aplicação.

Exemplo básico

```
server {  
    listen 80;  
    server_name localhost;  
  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://backend:3000/;  
    }  
}
```

Explicação

- **listen** → define a porta
- **location** → define como as rotas são tratadas
- **proxy_pass** → redirecionar requisições para outro serviço

4. Conceito sobre Mapeamento de Portas

No Docker, portas são mapeadas no formato:

```
host_port:container_port
```

Exemplo

```
ports:  
- "8080:80"
```

- Porta **8080** → acessada no navegador
- Porta **80** → porta interna do contêiner

Exercícios de fixação

- Crie um Dockerfile para uma aplicação simples.
- Crie um docker-compose.yml para uma aplicação que utilize MySQL.
- Configure um nginx.conf para servir arquivos estáticos e realizar proxy para uma API.
- Explique, com suas palavras, como funciona o mapeamento de portas no Docker.